Python List • Set • Dictionary

DSA & LeetCode Cheat Sheet — methods, built-ins, traversal, patterns, complexity, common mistakes, and the exact HashMap ideas used in our recent problems.

Core mental model

Structure	Stores	Order	Duplicates	Access
List	items	Keeps order	Allowed	By index
Set	unique items	No positional indexing	Not allowed	Membership
Dict / HashMap	key → value pairs	Insertion order preserved	Keys unique	By key

Quick decision guide

Use a list when you need sequence/indexing. Use a set when you mainly need uniqueness or fast membership. Use a dict when you need to associate one thing with another, such as number → latest index.

Python basics you should remember

```
# Create
a = [1, 2, 3]
s = {1, 2, 3}
d = {"a": 10, "b": 20}

# Empty structures
a = []
s = set() # NOT {}
d = {}

# Type
type(a), type(s), type(d)
```

Important: {} creates an empty dictionary, not an empty set. For an empty set use set().

1. LIST — Complete Cheat Sheet

A list is an ordered, mutable collection. It can contain duplicates and mixed data types.

Create / access / update

```
nums = [10, 20, 30, 40]

nums[0] # 10
nums[-1] # 40
nums[1:3] # [20, 30]
nums[::-1] # reversed copy

nums[1] = 99 # update
del nums[1] # delete by index

len(nums)
```

List methods

Method	Syntax	What it does
append	a.append(x)	Adds x at the end
extend	a.extend(iterable)	Adds all elements from iterable
insert	a.insert(i, x)	Inserts x before index i
remove	a.remove(x)	Removes first occurrence of x
pop	a.pop() / a.pop(i)	Removes and returns last / index i
clear	a.clear()	Removes all elements
index	a.index(x)	Returns first index of x; ValueError if absent
count	a.count(x)	Counts occurrences of x
sort	a.sort()	Sorts list in place
reverse	a.reverse()	Reverses list in place
copy	b = a.copy()	Shallow copy

Built-in functions commonly used with lists

Function	Example	Purpose
len	len(a)	Number of elements
sum	sum(a)	Sum of numeric elements
min	min(a)	Smallest element
max	max(a)	Largest element
sorted	sorted(a)	Returns a new sorted list
reversed	list(reversed(a))	Reverse iterator
enumerate	for i, x in enumerate(a)	Index + value
any	any(a)	True if at least one truthy item
all	all(a)	True if every item is truthy
list	list(iterable)	Creates a list

List traversal patterns

```
# Values
for x in nums:
print(x)

# Indices
for i in range(len(nums)):
print(i, nums[i])

# Index + value (preferred when both are needed)
for i, x in enumerate(nums):
print(i, x)

# Reverse traversal
for i in range(len(nums) - 1, -1, -1):
print(nums[i])

# Pair / adjacent traversal
```

```
for i in range(len(nums) - 1):
print(nums[i], nums[i + 1])
```

List comprehensions

```
squares = [x*x for x in nums]
evens = [x for x in nums if x % 2 == 0]
lower = [ch.lower() for ch in "Hello"]
```

List pitfalls

- append(x) adds one object; extend(x) adds elements from an iterable.
- sort() changes the list and returns None; sorted() returns a new list.
- remove(x) removes by value; pop(i) removes by index.
- Valid last index is len(a)-1, not len(a).
- str(list) does NOT join list elements into one normal string. Use ''.join(list_of_strings).

2. SET — Complete Cheat Sheet

A set stores unique, hashable elements. It is mainly useful for membership tests, removing duplicates, and set operations.

Create / add / remove

```
s = {1, 2, 3}
s = set() # empty set

s.add(4) # {1,2,3,4}
s.update([5, 6]) # add multiple

s.remove(2) # error if 2 is absent
s.discard(2) # no error if 2 is absent
x = s.pop() # removes and returns an arbitrary element
s.clear() # empty set
```

Set methods & operations

Method / operator	Example	Meaning
add	s.add(x)	Add one element
update	s.update(iterable)	Add many elements
remove	s.remove(x)	Remove x; KeyError if absent
discard	s.discard(x)	Remove x; no error if absent
pop	s.pop()	Remove/return arbitrary element
clear	s.clear()	Remove all
union	a | b / a.union(b)	Elements in either set
intersection	a & b	Elements common to both
difference	a - b	In a but not b
symmetric diff	a ^ b	In either, but not both
subset	a <= b / a.issubset(b)	Every a element is in b
superset	a >= b / a.issuperset(b)	Every b element is in a
isdisjoint	a.isdisjoint(b)	No common elements

Set traversal

```
s = {10, 20, 30}

for x in s:
print(x)

if 20 in s:
print("found")

# There is no reliable positional access like s[0].
# If you need indexing:
for i, x in enumerate(s):
print(i, x)
```

Do not depend on set iteration order. If you need a predictable order, use sorted(s).

Set conversions

```
nums = [1, 2, 2, 3, 3, 3]
unique = set(nums) # {1,2,3}

back_to_list = list(unique)

chars = set("banana") # unique characters
```

Common set patterns

```
# Duplicate detection
if x in seen:
print("duplicate")
seen.add(x)

# Remove duplicates
nums = list(set(nums))

# Common elements
common = set(a) & set(b)
```

```
# Check whether all required characters exist
required = set("balloon")
if required <= set(text):
print("possible")
```

3. DICTIONARY / HASH MAP — Complete Cheat Sheet

A dictionary stores key → value mappings. Keys must be hashable and unique. Dictionaries are the main Python tool for HashMap-style DSA problems.

Create / access / update

```
h = {"a": 10, "b": 20}

h["a"] # 10
h["c"] = 30 # add
h["a"] = 99 # update
del h["b"] # delete key

# Safe lookup
h.get("x") # None if missing
h.get("x", 0) # 0 if missing
```

Dictionary methods

Method	Syntax	What it does
get	d.get(k, default)	Returns value or default
keys	d.keys()	View of keys
values	d.values()	View of values
items	d.items()	(key, value) pairs
update	d.update(other)	Adds/overwrites pairs
pop	d.pop(k)	Removes and returns value
popitem	d.popitem()	Removes/returns last inserted pair
setdefault	d.setdefault(k, default)	Gets value; inserts default if missing
clear	d.clear()	Removes all pairs
copy	d.copy()	Shallow copy

Dictionary traversal

```
d = {"a": 10, "b": 20, "c": 30}

# Keys
for key in d:
print(key)

# Same as above
for key in d.keys():
print(key)

# Values
for value in d.values():
print(value)

# Key + value (MOST COMMON)
for key, value in d.items():
print(key, value)
```

Dictionary membership — very important

```
d = {"a": 10, "b": 20}

"a" in d # True — checks KEYS
10 in d # False — does not check values

10 in d.values() # True — checks VALUES
```

For normal HashMap lookup, prefer key in d over key in d.keys(). Both are valid, but direct dictionary membership expresses the intent more clearly.

Dictionary patterns for DSA

```
# Frequency map
freq = {}
for x in nums:
freq[x] = freq.get(x, 0) + 1

# Number -> latest index
h = {}
for i in range(len(nums)):
```

```
if nums[i] in h:
previous_index = h[nums[i]]
h[nums[i]] = i

# Grouping
groups = {}
for x in nums:
if x not in groups:
groups[x] = []
groups[x].append(x)

# Using setdefault for grouping
groups.setdefault(x, []).append(x)
```

The key pattern from Contains Duplicate II

```
nums = [1, 0, 1, 1]
k = 1
h = {}

for i in range(len(nums)):
if nums[i] in h and i - h[nums[i]] <= k:
return True

h[nums[i]] = i

return False
```

The design is number → latest index. That lets you ask directly: “Have I seen this number, and where was its most recent occurrence?”

4. String + Collection Methods You Need for DSA

split / strip / join

Method	Example	Result / use
split	"a b c".split()	['a','b','c']
split(separator)	"a,b,c".split(",")	['a','b','c']
strip	" hello ".strip()	"hello"
join	"-".join(["a","b"])	"a-b"

```
sentence = "Hello, World! How are you?"

words = sentence.split()
# ['Hello,', 'World!', 'How', 'are', 'you?']

clean = ""
for ch in sentence:
if ch.isalnum():
clean += ch.lower()

# Useful for Valid Palindrome:
# "A man, a plan, a canal: Panama" -> "amanaplanacanalpanama"
```

Character/string checks

Method	Meaning
.isalpha()	Only alphabetic characters
.isdigit()	Only decimal digit characters
.isalnum()	Alphabetic OR numeric
.isspace()	Whitespace
.lower()	Lowercase
.upper()	Uppercase
.startswith(x)	Starts with x
.endswith(x)	Ends with x
.count(x)	Number of non-overlapping occurrences
.find(x)	Index of first occurrence, -1 if absent

Useful built-ins for all three

Built-in	Typical use
len(x)	Number of elements/characters
sorted(x)	Sorted new list
enumerate(x)	Index + element
range(n)	Generate index range
zip(a,b)	Traverse two iterables together
reversed(x)	Reverse iterator
any(x)	At least one truthy item
all(x)	Every item truthy
list(x)	Convert to list
set(x)	Convert to set / remove duplicates
dict(x)	Create dictionary from suitable pairs
abs(x)	Absolute value; useful for index distance

Converting list ↔ string

```
chars = ['H', 'e', 'l', 'l', 'o']

s = ''.join(chars) # "Hello"
s = '-'.join(chars) # "H-e-l-l-o"

# NOT this:
str(chars) # "['H', 'e', 'l', 'l', 'o']"
```

5. Time Complexity — What Matters in LeetCode

Average-case complexity is the useful mental model for Python's hash-based structures.

Operation	List	Set	Dict
Access by index/key	O(1) index	—	O(1) avg key
Membership x in	O(n)	O(1) avg	O(1) avg key
Append/add	O(1) amortized	O(1) avg	O((1) avg insert
Delete by value/key	O(n) value	O(1) avg	O(1) avg
Search	O(n)	O(1) avg	O(1) avg by key
Sort	O(n log n)	sorted(s): O(n log n)	sorted(keys): O(n log n)

Choose the structure by the operation

Need	Usually choose
Keep order + index	List
Check whether something exists repeatedly	Set
Count frequencies	Dict
Value → latest index	Dict
Remove duplicates	Set
Key → associated data	Dict
Stack (simple)	List with append/pop
Queue	collections.deque

Classic DSA patterns

```
# 1. Seen set
seen = set()
for x in nums:
if x in seen:
return True
seen.add(x)

# 2. Frequency map
freq = {}
for x in nums:
freq[x] = freq.get(x, 0) + 1

# 3. Latest index map
last = {}
for i, x in enumerate(nums):
if x in last and i - last[x] <= k:
return True
last[x] = i

# 4. Prefix/suffix arrays
pre = [0] * len(nums)
suf = [0] * len(nums)

# 5. Two pointers
l, r = 0, len(nums) - 1
while l < r:
...
```

Common mistakes checklist

1. Index error: last index is len(a)-1.
2. Empty set: use set(), not {}.
3. Dict direction: decide what you need to look up. For Contains Duplicate II use number → index.
4. Missing dict key: use d.get(key, 0) when a default is appropriate.
5. List search: x in list is O(n); if you need repeated membership checks, consider a set.
6. Set indexing: s[0] is invalid.
7. sort vs sorted: sort() modifies; sorted() returns a new list.
8. remove vs pop: remove(value); pop(index) for lists.
9. Loop variable: after for ch in s, ch is one element, not the whole string.
10. Accumulator name: avoid naming a variable sum, list, set, or dict because they are built-ins.

6. One-Page Quick Reference

Task	Code
Create list	a = [1, 2, 3]
Create set	s = {1,2,3}; empty: set()
Create dict	d = {'a': 1}
List length	len(a)
List index	a[i]
List slice	a[l:r]
List add one	a.append(x)
List add many	a.extend(xs)
List remove value	a.remove(x)
List remove index	a.pop(i)
List sort	a.sort() / sorted(a)
Set membership	x in s
Set add	s.add(x)
Set remove safely	s.discard(x)
Set union	a | b
Set intersection	a & b
Set difference	a - b
Dict lookup	d[k]
Safe dict lookup	d.get(k, default)
Dict membership	k in d
Dict keys	d.keys()
Dict values	d.values()
Dict pairs	d.items()
Dict add/update	d[k] = v
Delete dict key	del d[k] / d.pop(k)
Loop values	for x in a/s:
Loop dict keys	for k in d:
Loop dict pairs	for k,v in d.items():
Index + value	for i,x in enumerate(a):
Reverse range	range(len(a)-1, -1, -1)
Absolute distance	abs(i-j)
String → list	s.split()
List[str] → string	''.join(chars)
Remove non-alphanumeric	if ch.isalnum(): ...

Final mental model

LIST = sequence → “I care about order and positions.”
SET = uniqueness → “Have I seen this before?”
DICT = relationship / lookup → “For this key, give me its value.”

For DSA: if you repeatedly search a list for something, ask yourself whether a set or dict can turn the search into a fast lookup.